



বরেন্দ্র বিশ্ববিদ্যালয়

VARENDRA UNIVERSITY



Department of Computer Science and Engineering

33rd Batch

Assignment

Mark	
------	--

Course title: Object Oriented Design and Design Patterns

Course Code: CSE 2101

Submitted By ID: 232311314 Name: Md Mohaiminul Islam Sajib Sec: H 33 th Batch 3 rd Sem . B.Sc. in Computer Science and Engineering at Varendra University	Submitted To
	Md. Mahfujur Rahman (Lecturer) Dept. of CSE, Varendra University
Date of Submission: 04 th Dec 2024	Signed By _____ Course Teacher

Title: Design and Implementation of a Multi-Functional Scientific Calculator Using Java Swing.

Objective:

To design and develop an intuitive, user-friendly scientific calculator application using Java Swing, equipped with advanced mathematical functionalities. This project aims to combine aesthetic appeal with computational efficiency, delivering a versatile tool for solving both basic arithmetic and complex scientific computations. Through seamless integration of a graphical user interface and robust backend logic, the application seeks to enhance the user experience while ensuring accuracy, reliability, and scalability.

Introduction:

A calculator is an essential tool used for performing mathematical operations quickly and accurately. This project focuses on developing a scientific calculator using Java Swing, blending functionality with an intuitive graphical user interface(GUI). The calculator supports basic arithmetic and advanced operations like trigonometry, logarithms, permutations, combinations, and more. Designed for accuracy and efficiency, it combines robust backend logic with an aesthetically pleasing interface, making complex calculations accessible to users. Additionally, the program includes robust error handling to ensure reliability and accuracy during computations. This application demonstrates the power of Java in creating versatile, interactive tools.

Calculator Design:



Methodology:

Java Swing is a powerful GUI (Graphical User Interface) toolkit that is part of the Java Foundation Classes (JFC). It is used to create window-based applications with graphical interfaces. Swing provides a set of lightweight, platform-independent components (like buttons, text fields, and tables) that can be customized and combined to build interactive user interfaces. Swing is built on top of AWT (Abstract Window Toolkit) but offers a richer set of features, including advanced controls and better flexibility in designing components. It uses the **Model-View-Controller (MVC)** design pattern, which separates the user interface (view) from the underlying data (model) and logic (controller). This makes it easier to develop and maintain complex applications. Swing applications are platform-independent, meaning they can run on any platform that supports Java, thanks to Java's "Write Once, Run Anywhere" principle. It provides a consistent look and feel across different operating systems, and developers can also create custom themes for applications. Java Swing is widely used for building desktop applications due to its ease of use, rich set of components, and flexibility in creating complex user interfaces.

1. Requirement Analysis:

- Identify the essential features for a scientific calculator, including basic arithmetic operations, advanced functions (trigonometric, logarithmic, factorial, etc.), and user-friendly interaction through a graphical interface.
- Define usability goals to ensure the application meets the needs of diverse users, such as students and professionals.

2. Design Phase:

- Create a structured design for the calculator using UML diagrams to outline components like the user interface and backend logic.
- Develop a graphical user interface (GUI) layout using Java Swing components like JFrame, JButton, and JTextField.

3. Development Phase:

- Implement the GUI to display buttons for operations and a display field for input and results.
- Write backend logic to handle mathematical computations, including a robust expression evaluator for operations involving parentheses, precedence, and functions.
- Integrate features such as factorials, permutations (nPr), combinations (nCr), and trigonometric functions.

4. Error Handling and Validation:

- Implement error handling for invalid inputs (e.g., division by zero, undefined trigonometric values).
- Validate user inputs to prevent syntax errors and ensure smooth operation.

5. Testing and Debugging:

- Test the calculator with diverse inputs to ensure accuracy and functionality.
- Debug and refine the code for edge cases and potential failures.

6. Deployment and User Experience Enhancement:

- Deploy the calculator as a standalone Java application.
- Enhance the user experience by adding features like color-coded buttons, themes, and a toggle for enabling/disabling the calculator.

7. Documentation:

- Prepare comprehensive documentation detailing the design, implementation, and usage of the calculator for end-users and developers.

This methodology ensures a systematic approach to developing a reliable, efficient, and aesthetically pleasing scientific calculator.

Procedure: Button Functions and Exception Handling

The scientific calculator implemented in this project has several buttons, each associated with specific mathematical operations. Below is a detailed explanation of each button's functionality, along with code snippets and how exceptions are handled for each operation.

1. Number Buttons (0-9)

Functionality:

Each number button appends its respective digit to the display screen. When pressed, it adds the corresponding number to the existing expression.

Code Snippet:

```
private void ZeroActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "0");  
}
```

Exception Handling:

There are no specific exceptions for number buttons, as they simply append digits to the display.

2. Decimal Point (.)

Functionality:

The dot button appends a decimal point to the display for floating-point operations.

Code Snippet:

```
private void DotActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + ".");  
}
```

Exception Handling:

Ensure that the dot is only added if the current input doesn't already have a decimal point to prevent multiple dots in a number.

```
if (!display.contains(".")) {  
    jTextField1.setText(display + ".");  
}
```

```
}
```

3. Basic Arithmetic Buttons (+, -, ×, ÷)

Summation (-) Button:

Functionality:

These buttons perform the basic arithmetic operations: addition, subtraction, multiplication, and division. They append the respective operator to the display for calculation.

Code Snippet for Plus (+):

```
private void PlusActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "+");  
    sign = sign + "+";  
}
```

Exception Handling:

Handle invalid inputs like consecutive operators or starting the expression with an operator.

```
if (display.isEmpty() || "+-*/" .contains(display.substring(display.length() -  
1))) {  
    // Error: Invalid expression  
    jTextField1.setText("Syntax Error");  
}
```

Subtraction (-) Button:

Functionality:

The "-" button performs subtraction and appends the operator to the display for calculation.

Code Snippet for Subtraction (-):

```
private void MinusActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "-");  
    sign = sign + "-";  
}
```

Exception Handling:

Ensure that the expression is valid, e.g., no consecutive operators or starting with a minus sign unless it's the first character.

```
if (display.isEmpty() || "+-*/".contains(display.substring(display.length() - 1))) {  
    // Error: Invalid expression  
    jTextField1.setText("Syntax Error");  
}
```

Multiplication (×) Button:

Functionality:

The "×" button performs multiplication and appends the operator to the display for calculation.

Code Snippet for Multiplication (×):

```
private void MultiplyActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "×");  
    sign = sign + "*"; // Internal representation can use "*" for  
    multiplication  
}
```

Exception Handling:

Check for invalid inputs like consecutive operators or starting the expression with a multiplication operator.

```
if (display.isEmpty() || "+-*/".contains(display.substring(display.length() - 1))) {  
    // Error: Invalid expression  
    jTextField1.setText("Syntax Error");  
}
```

Division (÷) Button:

Functionality:

The "÷" button performs division and appends the operator to the display for calculation.

Code Snippet for Division (÷):

```
private void DivideActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "÷");  
    sign = sign + "/"; // Internal representation can use "/" for division  
}
```

Exception Handling:

Ensure division by zero is handled properly and prevent consecutive operators.

```
if (display.isEmpty() || "+-*/".contains(display.substring(display.length() -  
1))) {  
    // Error: Invalid expression  
    jTextField1.setText("Syntax Error");  
} else if (display.endsWith("÷0")) {  
    // Error: Division by zero  
    jTextField1.setText("Math Error: Div by 0");  
}
```

4. Equal Button (=)

Functionality:

The equal button evaluates the expression entered in the display and shows the result.

Code Snippet:

```
private void EqualActionPerformed(java.awt.event.ActionEvent evt) {  
    try {  
        String expression = jTextField1.getText().trim();  
        double result = evaluate(expression);  
        lastAnswer = result;  
        jTextField1.setText(String.valueOf(result));  
    } catch (ArithmeticException e) {  
        jTextField1.setText("Math Error: " + e.getMessage());  
    } catch (IllegalArgumentException e) {  
        jTextField1.setText("Syntax Error: " + e.getMessage());  
    } catch (Exception e) {  
        jTextField1.setText("Error");  
    }  
}
```

Exception Handling:

- **ArithmeticException:** Handles mathematical errors like division by zero.
 - **IllegalArgumentException:** Catches syntax errors (e.g., malformed expressions).
 - **Generic Exception:** Catches any other unexpected errors.
-

5. Clear (AC) Button

Functionality:

The AC (All Clear) button resets the display to an empty state, clearing any previous inputs or results.

Code Snippet:

```
private void AcActionPerformed(java.awt.event.ActionEvent evt) {  
    display = "";  
    jTextField1.setText(display);  
    a = null;  
    sign = "";  
    result = 0;  
}
```

Exception Handling:

No specific exception handling needed, as it just resets the state of the calculator.

6. Backspace (DEL) Button

Functionality:

The DEL (Delete) button removes the last character from the display, allowing the user to correct their input.

Code Snippet:

```
private void DelActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    if (display.length() > 0) {  
        display = display.substring(0, display.length() - 1);  
        jTextField1.setText(display);  
    }  
}
```


Exception Handling:

If the display is empty, no operation should occur. This is handled by checking the length before modifying the string.

7. Trigonometric Functions (Sin, Cos, Tan, etc.)

Sine Function (Sin):

Functionality:

These buttons perform trigonometric operations. For example, the "sin" button computes the sine of a number (in degrees).

Code Snippet for Sin:

```
private void SinActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "sin(");  
}
```

Exception Handling:

Ensure the input within the parentheses is valid and handle edge cases (e.g., inputting sin(90) directly).

```
if (expression.startsWith("sin(") && expression.endsWith("))") {  
    try {  
        String inside = expression.substring(4, expression.length() -  
1).trim();  
        double result = evaluate(inside);  
        return Math.sin(Math.toRadians(result)); // sin expects input in  
radians  
    } catch (Exception e) {  
        throw new ArithmeticException("Invalid input for sin function");  
    }  
}
```

Cosine Function (Cos):

Functionality:

The "cos" button computes the cosine of a given number (in degrees).

Code Snippet for Cos:

```
private void CosActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "cos(");  
}
```

Exception Handling:

Ensure valid input within the parentheses and handle edge cases (e.g., inputting cos(90) directly, where cos(90) = 0).

```
if (expression.startsWith("cos(") && expression.endsWith("))") {  
    try {  
        String inside = expression.substring(4, expression.length() -  
1).trim();  
        double result = evaluate(inside);  
        return Math.cos(Math.toRadians(result)); //cos expects input in  
radians  
    } catch (Exception e) {  
        throw new ArithmeticException("Invalid input for cos function");  
    }  
}
```

Tangent Function (Tan):

Functionality:

The "tan" button computes the tangent of a given number (in degrees).

Code Snippet for Tan:

```
private void TanActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "tan(");  
}
```

Exception Handling:

Handle special cases where tan(x) is undefined, such as for tan(90) or tan(270) (which results in infinity).

```
if (expression.startsWith("tan(") && expression.endsWith("))") {  
    try {
```

```

        String inside = expression.substring(4, expression.length() -
1).trim();
        double result = evaluate(inside);

        // Check for undefined tan values (tan(90), tan(270), etc.)
        if (result % 180 == 90) {
            throw new ArithmeticException("tan(" + result + ") is undefined
(infinity).");
        }
        return Math.tan(Math.toRadians(result)); // tan expects input in
radians
    } catch (Exception e) {
        throw new ArithmeticException("Invalid input for tan function");
    }
}

```

Cosecant Function (Cosec):

Functionality:

The "cosec" button computes the cosecant of a given number, which is $1/\sin(x)$.

Code Snippet for Cosec:

```

private void Sin_inActionPerformed(java.awt.event.ActionEvent evt) {
    display = jTextField1.getText();
    jTextField1.setText(display + "cosec(");
}

```

Exception Handling:

Ensure the sine of the value is not zero (as cosec(x) is undefined for $\sin(x) = 0$).

```

if (expression.startsWith("cosec(") && expression.endsWith("))") {
    try {
        String inside = expression.substring(6, expression.length() -
1).trim();
        double value = evaluate(inside); // Evaluate inside the cosec()
function
        double sineValue = Math.sin(Math.toRadians(value)); // sin(x) in
radians
        if (Math.abs(sineValue) < 1e-10) { // Avoid division by zero with a
small tolerance
            throw new ArithmeticException("cosec(" + value + ") is
undefined.");
        }
    }
}

```

```

    }
    return 1 / sineValue; // Return cosec(x) = 1 / sin(x)
} catch (Exception e) {
    throw new ArithmeticException("Invalid input for cosec function");
}
}

```

Secant Function (Sec):

Functionality:

The "sec" button computes the secant of a given number, which is $1/\cos(x)$.

Code Snippet for Sec:

```

private void Cos_inActionPerformed(java.awt.event.ActionEvent evt) {
    display = jTextField1.getText();
    jTextField1.setText(display + "sec(");
}

```

Exception Handling:

Ensure the cosine of the value is not zero (as $\sec(x)$ is undefined for $\cos(x) = 0$).

```

if (expression.startsWith("sec(") && expression.endsWith(")")) {
    try {
        String inside = expression.substring(4, expression.length() -
1).trim();
        double value = evaluate(inside); // Evaluate inside sec() function
        double cosineValue = Math.cos(Math.toRadians(value)); // cos(x) in
radians
        if (Math.abs(cosineValue) < 1e-10) { // Avoid division by zero with a
small tolerance
            throw new ArithmeticException("sec(" + value + ") is undefined.");
        }
        return 1 / cosineValue; // Return sec(x) = 1 / cos(x)
    } catch (Exception e) {
        throw new ArithmeticException("Invalid input for sec function");
    }
}

```

Cotangent Function (Cot):

Functionality:

The "cot" button computes the cotangent of a given number, which is $1/\tan(x)$.

Code Snippet for Cot:

```
private void Tan_inActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "cot(");  
}
```

Exception Handling:

Handle undefined values where $\cot(x)$ is infinite, such as for $\cot(90)$ or $\cot(270)$.

```
if (expression.startsWith("cot(") && expression.endsWith("))") {  
    try {  
        String inside = expression.substring(4, expression.length() -  
1).trim();  
        double value = evaluate(inside); // Evaluate inside cot() function  
        // Normalize the angle to handle periodicity  
        value = value % 360;  
        if (value < 0) {  
            value += 360; // Ensure the angle is positive  
        }  
  
        // Handle special cases for cot(x)  
        if (value == 90 || value == 270) {  
            return Double.POSITIVE_INFINITY; // cot(90) and cot(270) are  
infinity  
        }  
  
        double tangentValue = Math.tan(Math.toRadians(value));  
        if (Math.abs(tangentValue) < 1e-10) { // Handle near-zero tan(x)  
            throw new ArithmeticException("cot(" + value + ") is undefined.");  
        }  
        return 1 / tangentValue; // Compute cot(x) = 1 / tan(x)  
    } catch (Exception e) {  
        throw new ArithmeticException("Invalid input for cot function");  
    }  
}
```

8. Logarithmic and Exponential Functions (log, ln, e)

Logarithm Base 10 (log) Function:

Functionality:

The "log" button computes the logarithm of a number with base 10.

Code Snippet for Log:

```
private void LogActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText("log(" + display);  
}
```

Exception Handling:

Logarithms are undefined for values less than or equal to zero in real numbers. Handle these cases to ensure the calculator doesn't crash.

```
if (expression.startsWith("log(") && expression.endsWith("))") {  
    try {  
        String inside = expression.substring(4, expression.length() -  
1).trim();  
        double value = evaluate(inside);  
  
        // Logarithm is undefined for non-positive values  
        if (value <= 0) {  
            throw new ArithmeticException("Logarithm undefined for non-positive  
numbers");  
        }  
        return Math.log10(value); // Calculate log base 10  
    } catch (Exception e) {  
        throw new ArithmeticException("Invalid input for log function");  
    }  
}
```

Natural Logarithm (ln) Function

Functionality:

The "ln" button computes the natural logarithm (logarithm with base e) of a number.

Code Snippet for ln:

```
private void LnActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();
```

```
jTextField1.setText("ln(" + display);  
}
```

Exception Handling:

Similar to the log function, the natural logarithm is undefined for non-positive values. Ensure proper error handling for these cases.

```
if (expression.startsWith("ln(") && expression.endsWith(")")) {  
    try {  
        String inside = expression.substring(3, expression.length() -  
1).trim();  
        double value = evaluate(inside);  
  
        // Natural logarithm is undefined for non-positive values  
        if (value <= 0) {  
            throw new ArithmeticException("Natural logarithm undefined for non-  
positive numbers");  
        }  
        return Math.log(value); // Calculate natural logarithm  
    } catch (Exception e) {  
        throw new ArithmeticException("Invalid input for ln function");  
    }  
}
```

Euler's Number (e) Function

Functionality:

The "e" button appends the constant e (Euler's number) to the current expression. When used in calculations, e is treated as the mathematical constant, approximately equal to 2.71828.

Code Snippet for e:

```
private void EActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "e");  
}
```

Exception Handling:

There are no specific exceptions for the e button, as it simply appends the constant to the expression.

9. Power Functions (x^2 , x^3 , x^n , 2^n , $1/x$, $\sqrt{x}$, $\sqrt[3]{x}$, $(|x|)$)

Square Function (x^2):

Functionality:

These buttons compute powers like x^2 (square).

Code Snippet for x^2 :

```
private void Xpower2ActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "^2");  
}
```

Exception Handling:

Handle invalid inputs, especially for exponentiation, where the base or exponent might be non-numeric.

```
if (!isNumeric(base) || !isNumeric(exponent)) {  
    throw new ArithmeticException("Invalid number for power operation");  
}
```

Cube Function (x^3):

Functionality:

The " x^3 " button computes the cube of a number.

Code Snippet for x^3 :

```
private void Xpower3ActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "^3");  
}
```

Exception Handling:

Ensure valid input for exponentiation and handle non-numeric values.

```
if (!isNumeric(base)) {  
    throw new ArithmeticException("Invalid number for power operation");  
}
```



```
}  
return Math.pow(base, 3); // Compute x^3
```

Exponentiation Function (x^n):

Functionality:

The " x^n " button computes x raised to the power of n (i.e., x^n).

Code Snippet for x^n :

```
private void XpowerNActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "^");  
}
```

Exception Handling:

Ensure both the base and exponent are numeric and handle invalid input.

```
if (!isNumeric(base) || !isNumeric(exponent)) {  
    throw new ArithmeticException("Invalid number for power operation");  
}  
return Math.pow(base, exponent); // Compute x^n
```

Power of 2 (2^n):

Functionality:

The " 2^n " button computes 2 raised to the power of n (i.e., 2^n).

Code Snippet for 2^n :

```
private void Two_squareActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "2^");  
}
```

Exception Handling:

Handle invalid exponent values (non-numeric).

```
if (!isNumeric(exponent)) {  
    throw new ArithmeticException("Invalid exponent for 2^n");  
}
```

```
}  
return Math.pow(2, exponent); // Compute 2^n
```

Reciprocal Function (1/x):

Functionality:

The "1/x" button computes the reciprocal of a number (i.e., $1/x$).

Code Snippet for 1/x:

```
private void XinverseActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "1/(");  
}
```

Exception Handling:

Handle division by zero by checking the value of x before calculating the reciprocal.

```
if (x == 0) {  
    throw new ArithmeticException("Cannot divide by zero");  
}  
return 1 / x; // Compute 1/x
```

Square Root Function ($\sqrt{x}$):

Functionality:

The " $\sqrt{x}$ " button computes the square root of a number.

Code Snippet for Square Root ($\sqrt{x}$):

```
private void SqrtActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText("√(" + display + ")");  
}
```

Exception Handling:

Handle negative numbers for square root calculations, as square roots of negative numbers are undefined in real numbers.

```
if (x < 0) {
```

```
        throw new ArithmeticException("Square root of a negative number is  
undefined");  
    }  
    return Math.sqrt(x); // Compute  $\sqrt{x}$ 
```

Cube Root Function ($\sqrt[3]{x}$):

Functionality:

The " $\sqrt[3]{x}$ " button computes the cube root of a number.

Code Snippet for Cube Root ($\sqrt[3]{x}$):

```
private void Sqrt3ActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(" $\sqrt[3]{$ " + display + ")");  
}
```

Exception Handling:

Cube roots are defined for all real numbers, so there's no specific exception for negative numbers.

```
return Math.cbrt(x); // Compute cube root
```

Absolute Value Function ($|x|$):

Functionality:

The " $|x|$ " button computes the absolute value of a number.

Code Snippet for Absolute Value ($|x|$):

```
private void Sqrt4ActionPerformed(java.awt.event.ActionEvent evt) {  
    try {  
        display = jTextField1.getText();  
        double value = Double.parseDouble(display);  
        jTextField1.setText(String.valueOf(Math.abs(value)));  
    } catch (Exception e) {  
        jTextField1.setText("Error");  
    }  
}
```

Exception Handling:

Handle invalid inputs, such as non-numeric values.

```
try {  
    double value = Double.parseDouble(display);  
    return Math.abs(value); // Compute |x|  
} catch (NumberFormatException e) {  
    throw new ArithmeticException("Invalid input for absolute value");  
}
```

10. Factorial (x!) Button

Functionality:

This button calculates the factorial of a given number.

Code Snippet:

```
private void FactorialActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "!");  
}
```

Exception Handling:

Factorial is only defined for non-negative integers. Ensure the input is valid.

11. Percentage (%) Function:

Functionality:

The "%" button computes the percentage of a given number.

Code Snippet for %:

```
private void PercentageActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "%");  
}
```

Exception Handling:

Ensure that the percentage operation only happens when valid numeric input is provided.

```
if (!isNumeric(value)) {
```

```
        throw new ArithmeticException("Invalid input for percentage operation");
    }
    return value / 100; // Convert to percentage
}
```

12. Binary (BIN) Function:

Functionality:

The "BIN" button converts the input number to its binary representation.

Code Snippet for BIN:

```
private void BinActionPerformed(java.awt.event.ActionEvent evt) {
    display = jTextField1.getText();
    jTextField1.setText(display + "Bin(");
}
```

Exception Handling:

Binary conversion is only valid for integers. Handle cases where the input is not an integer.

```
if (!isNumeric(value) || value % 1 != 0) {
    throw new ArithmeticException("Binary conversion requires an integer");
}
int binaryResult = Integer.parseInt(String.valueOf(value));
return Integer.toBinaryString(binaryResult); // Convert to binary string
}
```

13. Answer (Ans) Function

Functionality:

The "Ans" button retrieves the most recent result (from the last calculation) and displays it.

Code Snippet for Ans:

```
private void AnsActionPerformed(java.awt.event.ActionEvent evt) {
    jTextField1.setText(String.valueOf(lastAnswer));
}
```

Exception Handling:

There's no specific exception for the **Ans** button, but ensure that the lastAnswer is not empty or invalid.

```
if (lastAnswer == null) {  
    throw new ArithmeticException("No previous answer to display");  
}
```

14. Exponentiation (EXP) Function:

Functionality:

The "EXP" button appends the exponent notation (E) for scientific notation or to represent powers of 10.

Code Snippet for EXP:

```
private void ExpActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "E");  
}
```

Exception Handling:

There are no specific exceptions for the **EXP** button, but ensure the input is formatted correctly before performing operations on it.

```
if (!isNumeric(display)) {  
    throw new ArithmeticException("Invalid input for exponential function");  
}
```

15. Parentheses ()

Functionality:

The open (and close) parentheses are used to group expressions for order of operations.

Code Snippet for Parentheses (Open and Close):

```
private void Bracket_openActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();  
    jTextField1.setText(display + "(");  
}  
  
private void Bracket_closeActionPerformed(java.awt.event.ActionEvent evt) {  
    display = jTextField1.getText();
```

```
jTextField1.setText(display + ")");  
}
```

Exception Handling:

Ensure that parentheses are balanced and that they appear in the correct order (e.g., not closing a parenthesis without an open one).

```
if (openParenthesesCount > closeParenthesesCount) {  
    throw new ArithmeticException("Mismatched parentheses");  
}
```

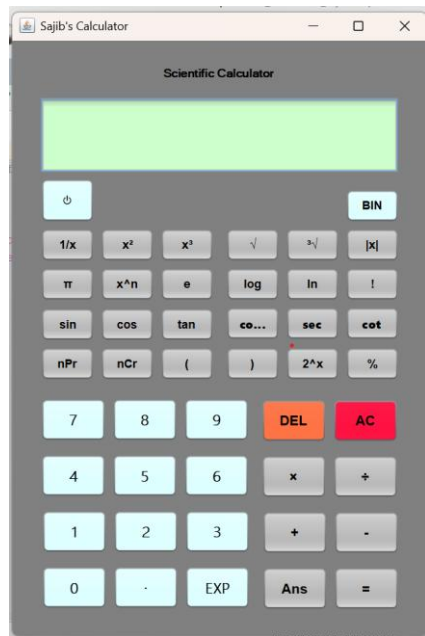
Handling General Exceptions in the Calculator:

For operations like evaluation, trigonometric calculations, and logarithmic computations, exceptions such as `ArithmeticException` (for division by zero), `IllegalArgumentException` (for invalid syntax), and generic `Exception` (for unforeseen errors) can be caught and handled as shown in the **Equal** button example.

General Exception Handling:

```
try {  
    String expression = jTextField1.getText().trim();  
    double result = evaluate(expression); // Custom method to evaluate the  
    expression  
    jTextField1.setText(String.valueOf(result));  
} catch (ArithmeticException e) {  
    jTextField1.setText("Math Error: " + e.getMessage());  
} catch (IllegalArgumentException e) {  
    jTextField1.setText("Syntax Error: " + e.getMessage());  
} catch (Exception e) {  
    jTextField1.setText("Error: " + e.getMessage());  
}
```

Calculator Demo:



Conclusion:

In this project, we developed a scientific calculator using Java Swing that supports a wide range of mathematical operations, from basic arithmetic to complex trigonometric, logarithmic, and power functions. The application combines a user-friendly graphical interface with robust backend logic, making it a versatile tool for various users. With careful exception handling, the calculator ensures reliable performance by addressing invalid inputs and mathematical errors. This project serves as a valuable example of creating interactive applications in Java, and I look forward to implementing similar projects in the future.